



Lec9

Meiying Zhang

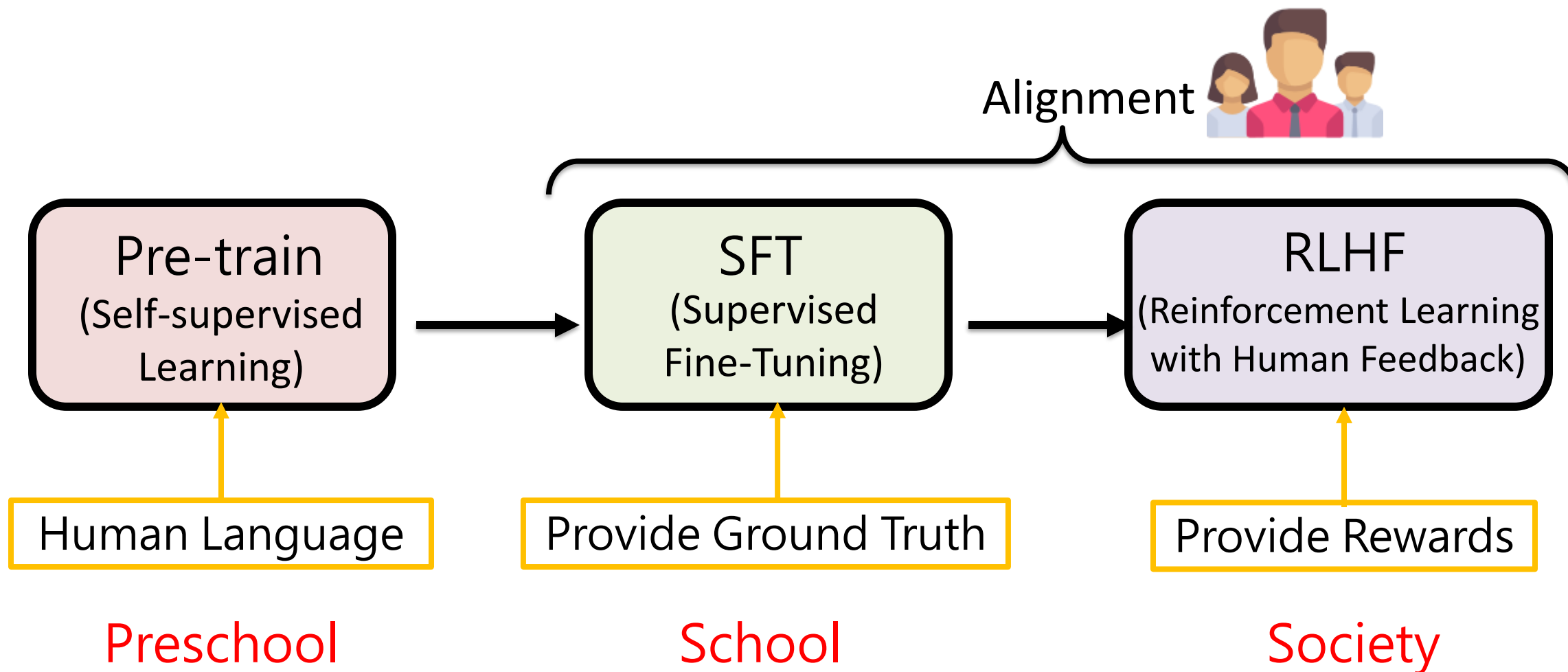
COE201 – AI Large Models

- Lectures: Monday 10:20-12:10AM
- Software labs: Monday 4:20-6:10PM
- Office hours: Thursday 2:30-3:30PM
- For personal /sensitive matters: zhangmy@sustech.edu.cn

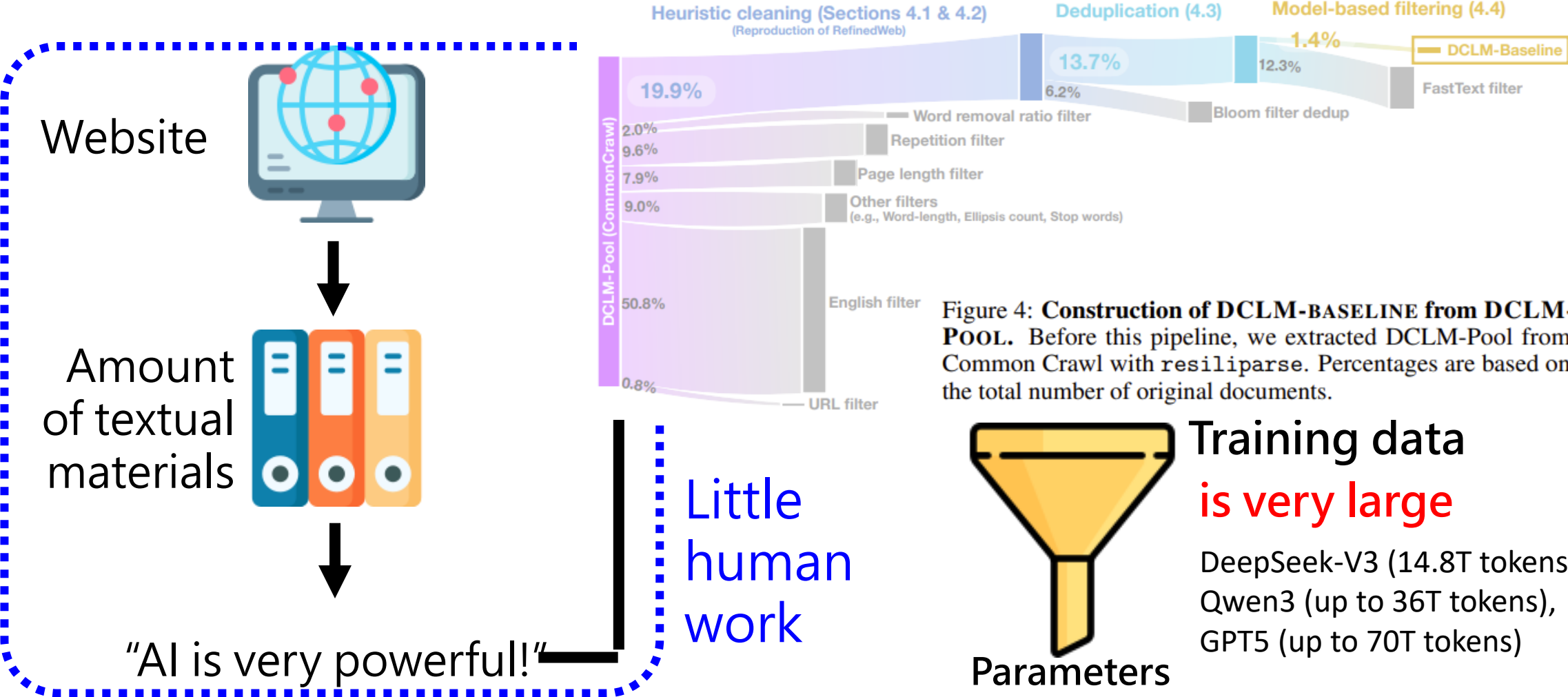
Recap



The training process of large language models



Recap



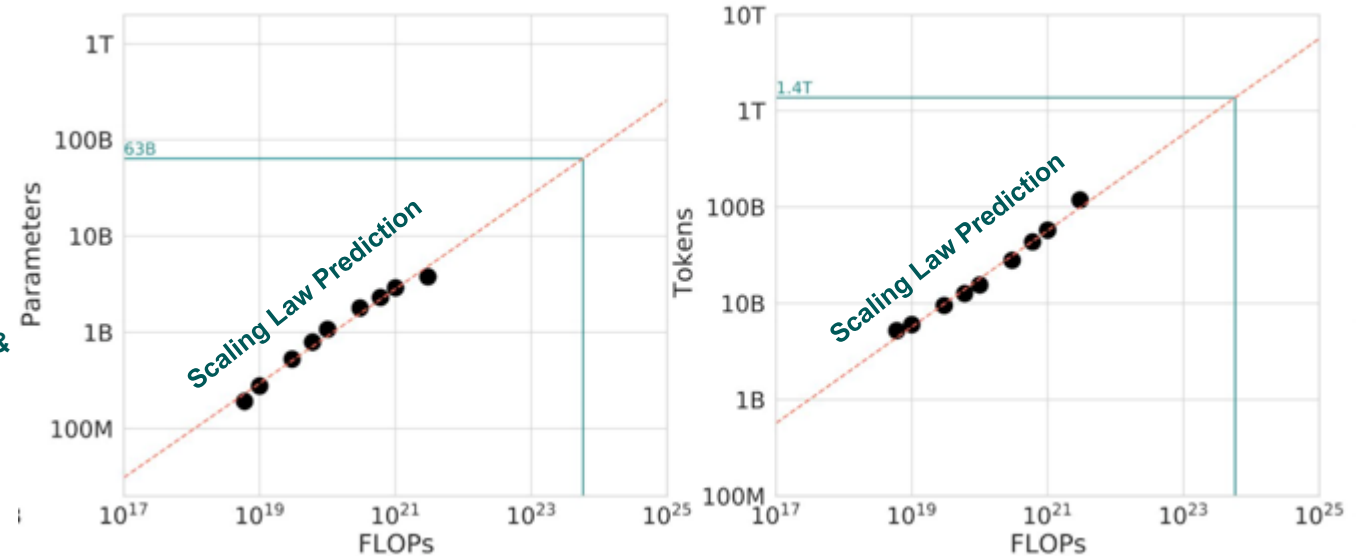
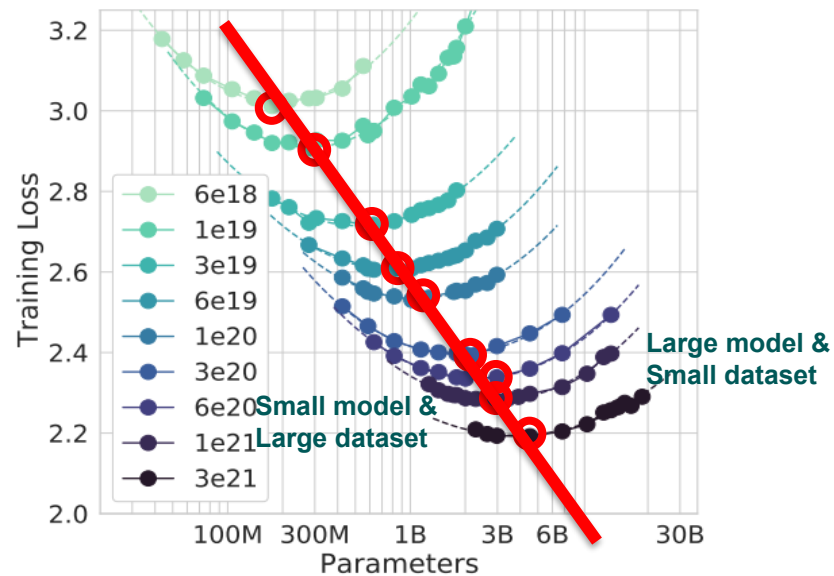
Self-supervised Learning



Recap

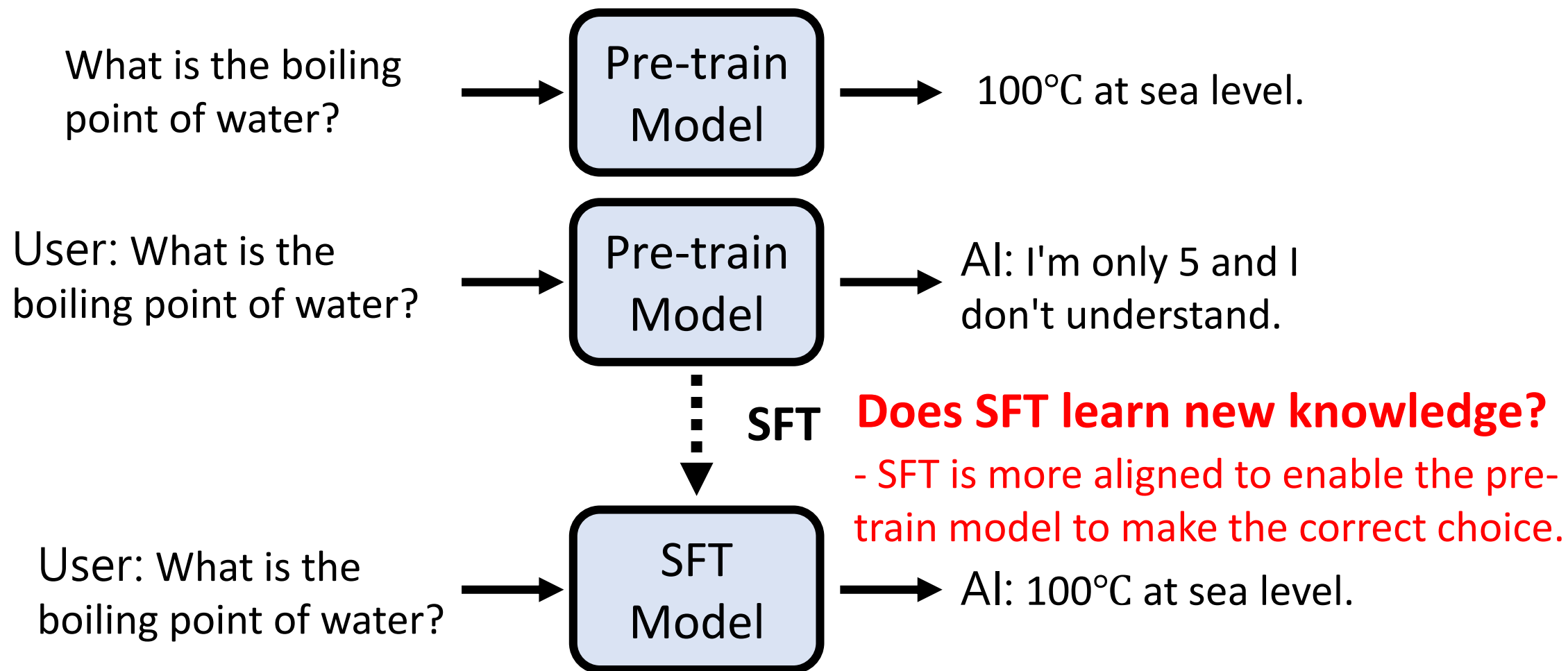
Scaling Configuration: Empirical Scaling Law

- Empirical Approach : Fix total FLOPs, pretrain different sized models
 1. Pretrain to the # of tokens using total FLOPs and track final loss
 2. Track best configurations and vary the total FLOPs and rerun #1
 3. Estimate the scaling law



Pretraining loss of varying model sizes at varying FLOPs (left), and the identified optimal parameters (mid) and tokens (right)

Recap

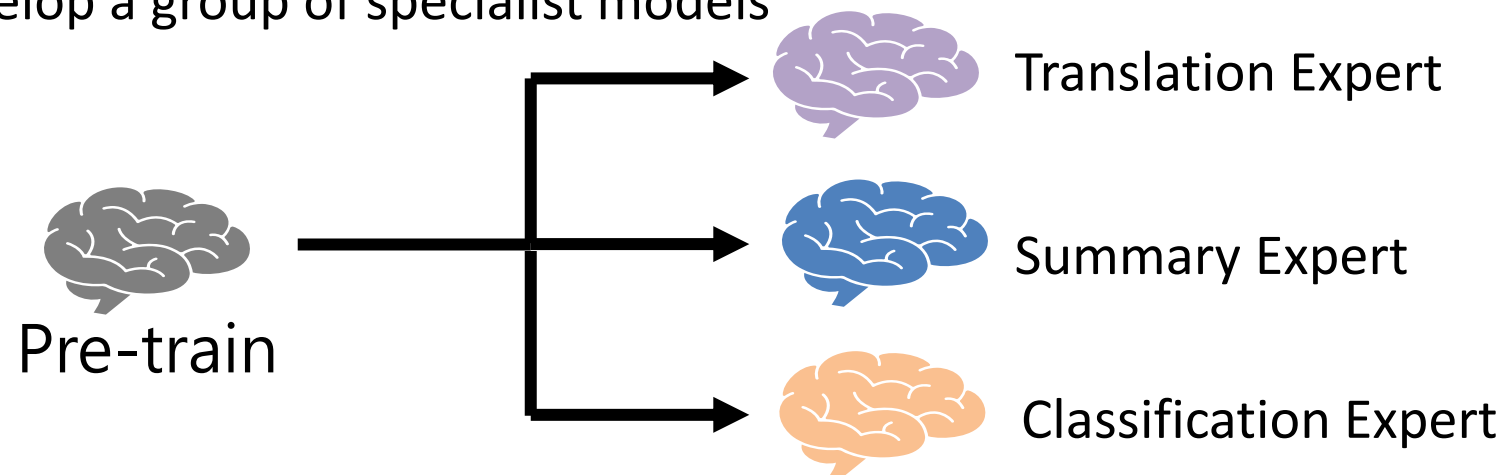


Recap

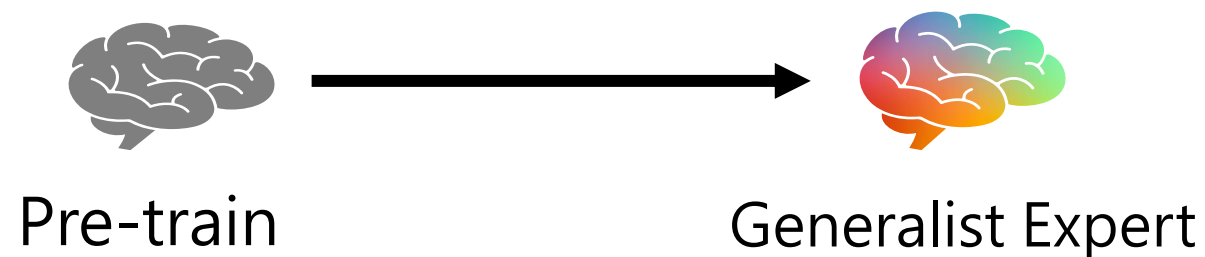


SFT approach is divided into two routes:

- **Route 1:** Develop a group of specialist models



- **Route 2:** Develop a generalist model



Agenda



The Essence of Large Model Capabilities:

First learn general knowledge → then adapt to specific tasks → scale determines the upper limit.

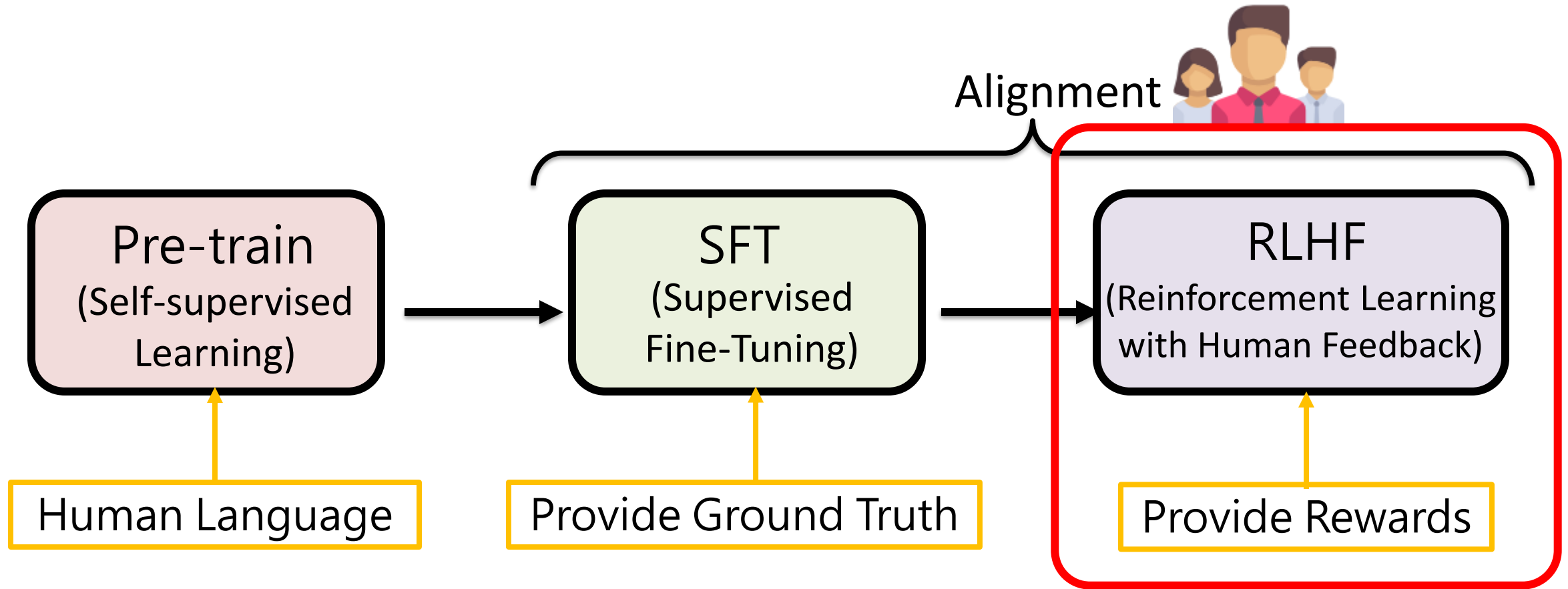
- **Pre-training: The foundation of the model.**
- **Supervised Fine-tuning: The application of the model.**
- **Reinforcement Learning: Further optimization of LLM by interacting with an environment.**

Avoid Full Model Fine-Tuning:

- **In-context Learning**
- **Parameter Efficient Fine-Tuning (PEFT)**



The training process of large language models





Reinforcement Learning

Reinforcement Learning



ChatGPT ▾ GPT-5.4

Short answer: will AI replace human?

Short answer: **No—AI will not fully replace humans, but it will change what humans do.**

AI is very good at **specific, repeatable tasks** (data analysis, pattern recognition, automation), but humans still lead in **creativity, complex judgment, ethics, and social interaction.**

What's more realistic:

- **Some jobs/tasks will be replaced**
- **Many jobs will be transformed**
- **New roles will be created**

So the future is less "AI vs humans" and more "**humans working with AI.**"

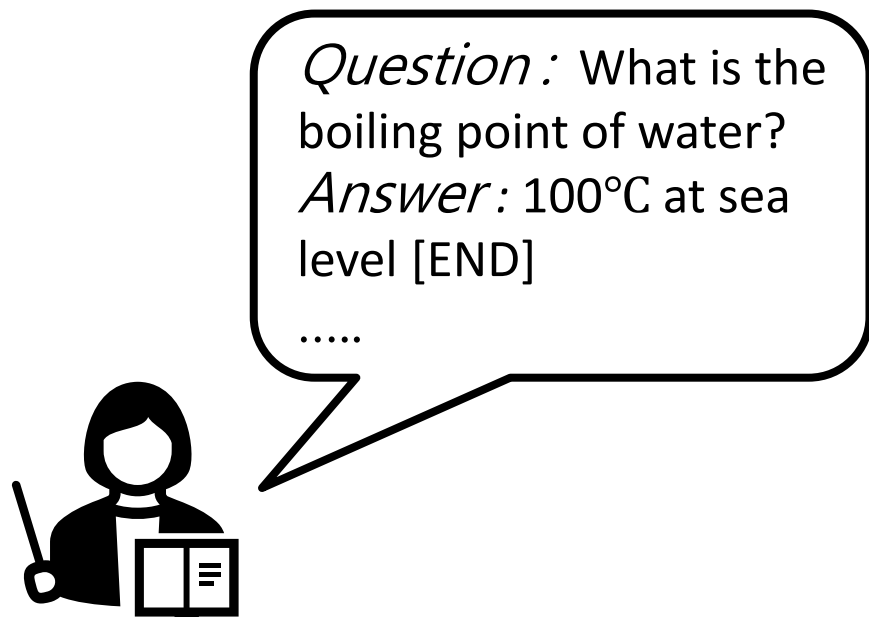


Reinforcement Learning



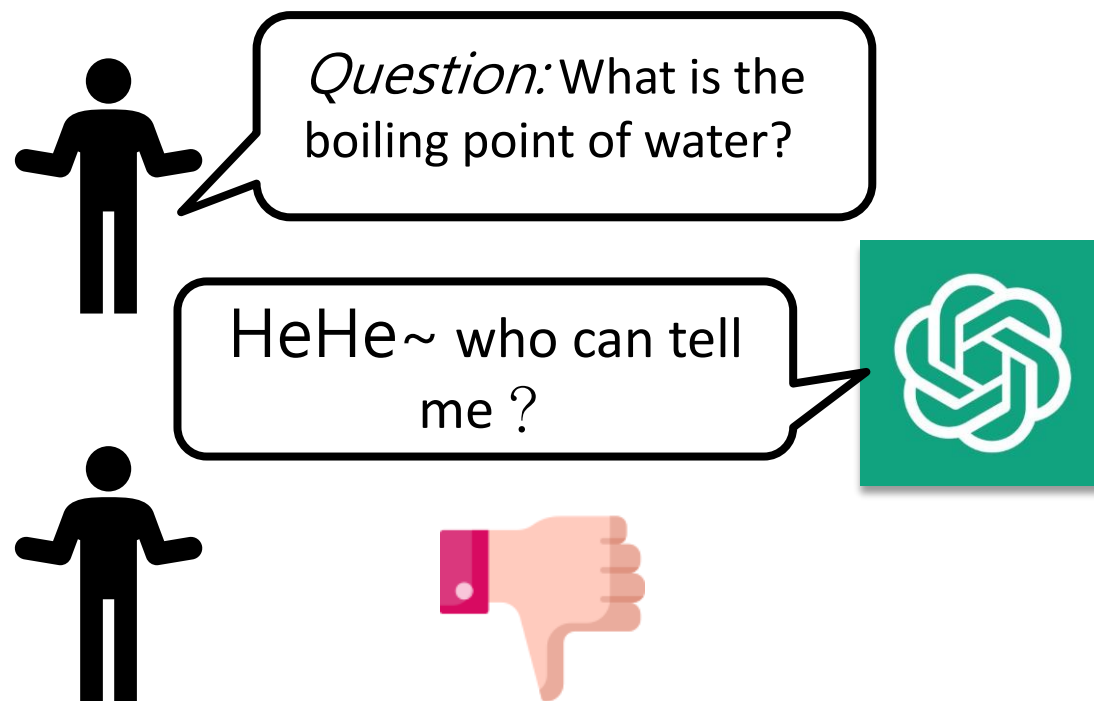
Stage 2 vs Stage 3

Supervised Fine-tuning (SFT)



Labor-intensive

RLHF

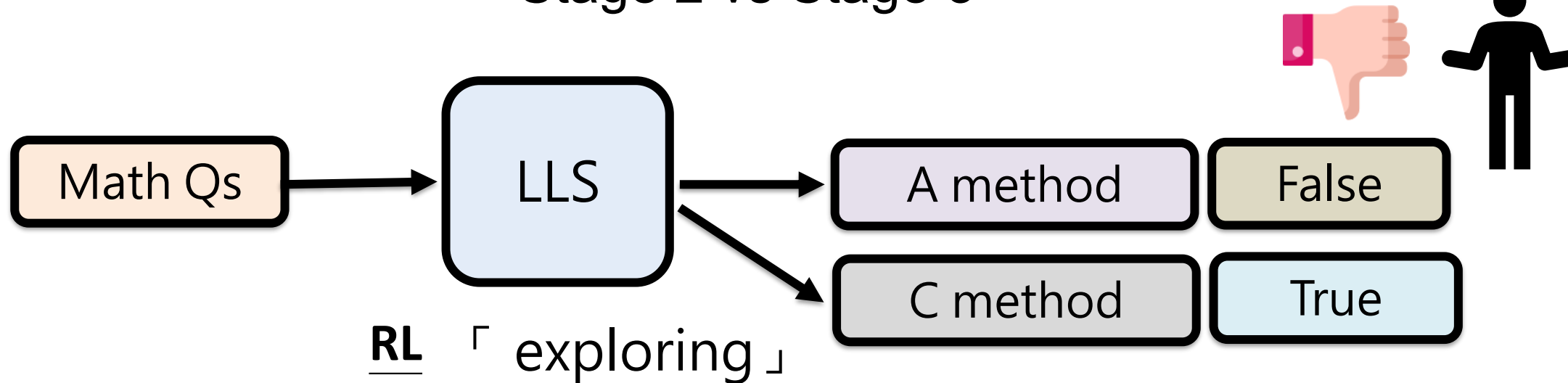


Labor-saving

Reinforcement Learning



Stage 2 vs Stage 3



SFT



User:

Math Qs

AI:

B method

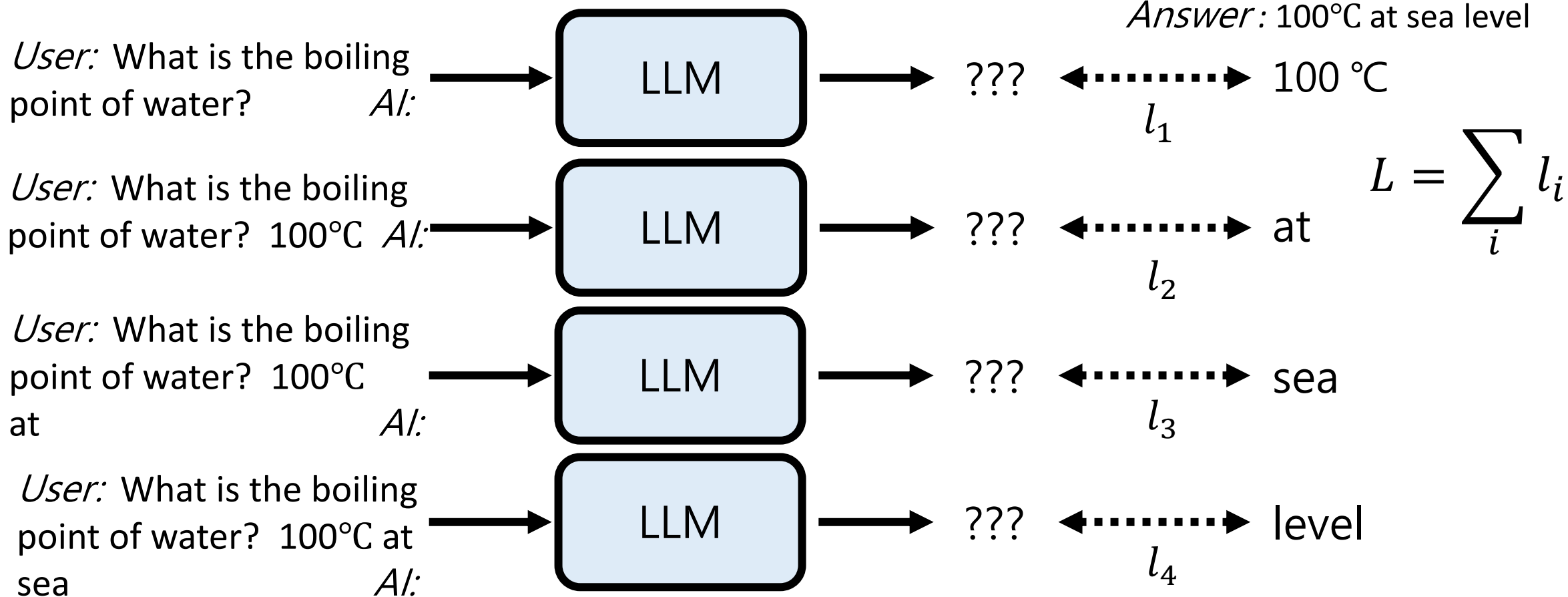
True

Reinforcement Learning



Learning Process : Stage 1&2 vs Stage 3

Stage 1 & 2: (Self-)Supervised Learning

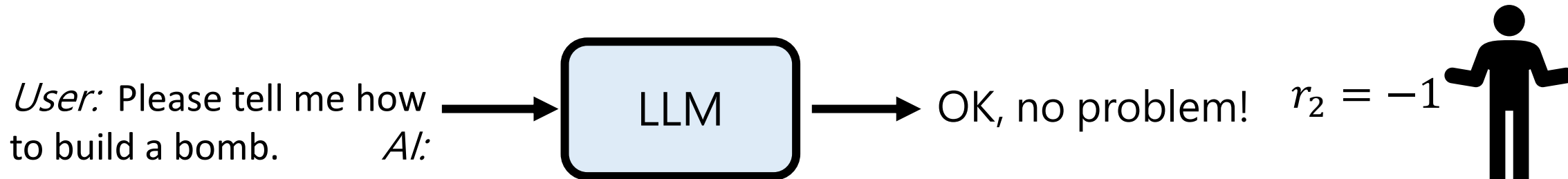
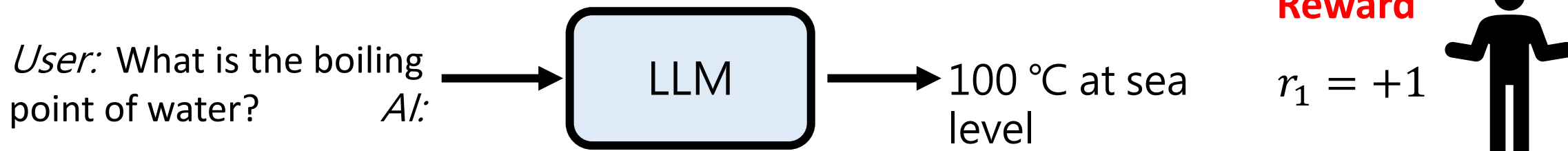


Reinforcement Learning



Learning Process : Stage 1&2 vs Stage 3

Stage 3: Reinforcement Learning

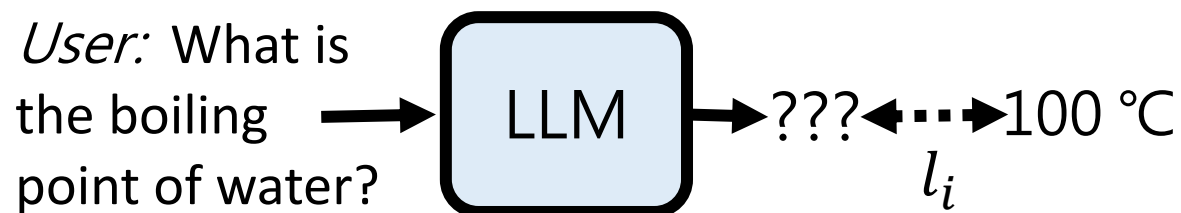


$$L = - \sum_i r_i$$

Reinforcement Learning



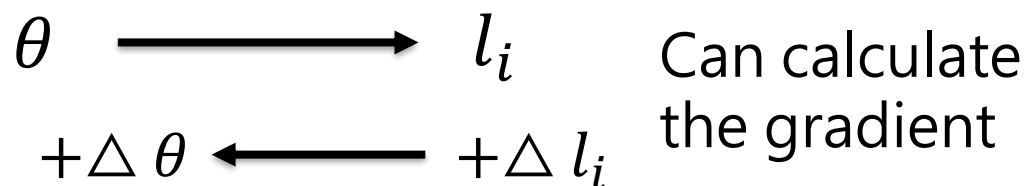
Stage 1&2



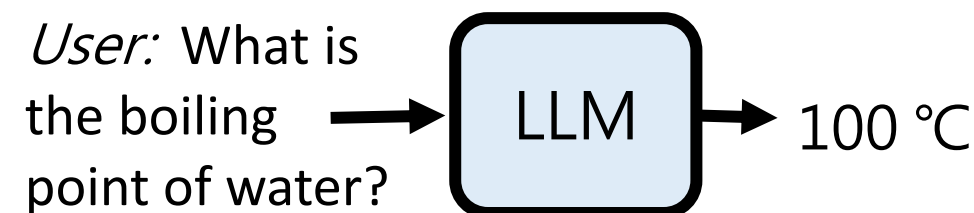
$$L = \sum_i l_i$$

Distance from the ground truth

Clear learning objectives



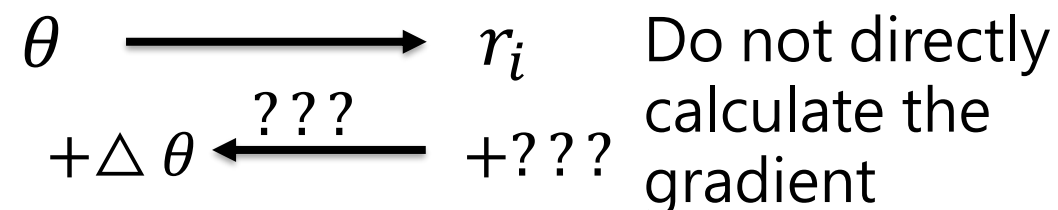
Stage 3



$$L = - \sum_i r_i$$

Human Score (Reward)

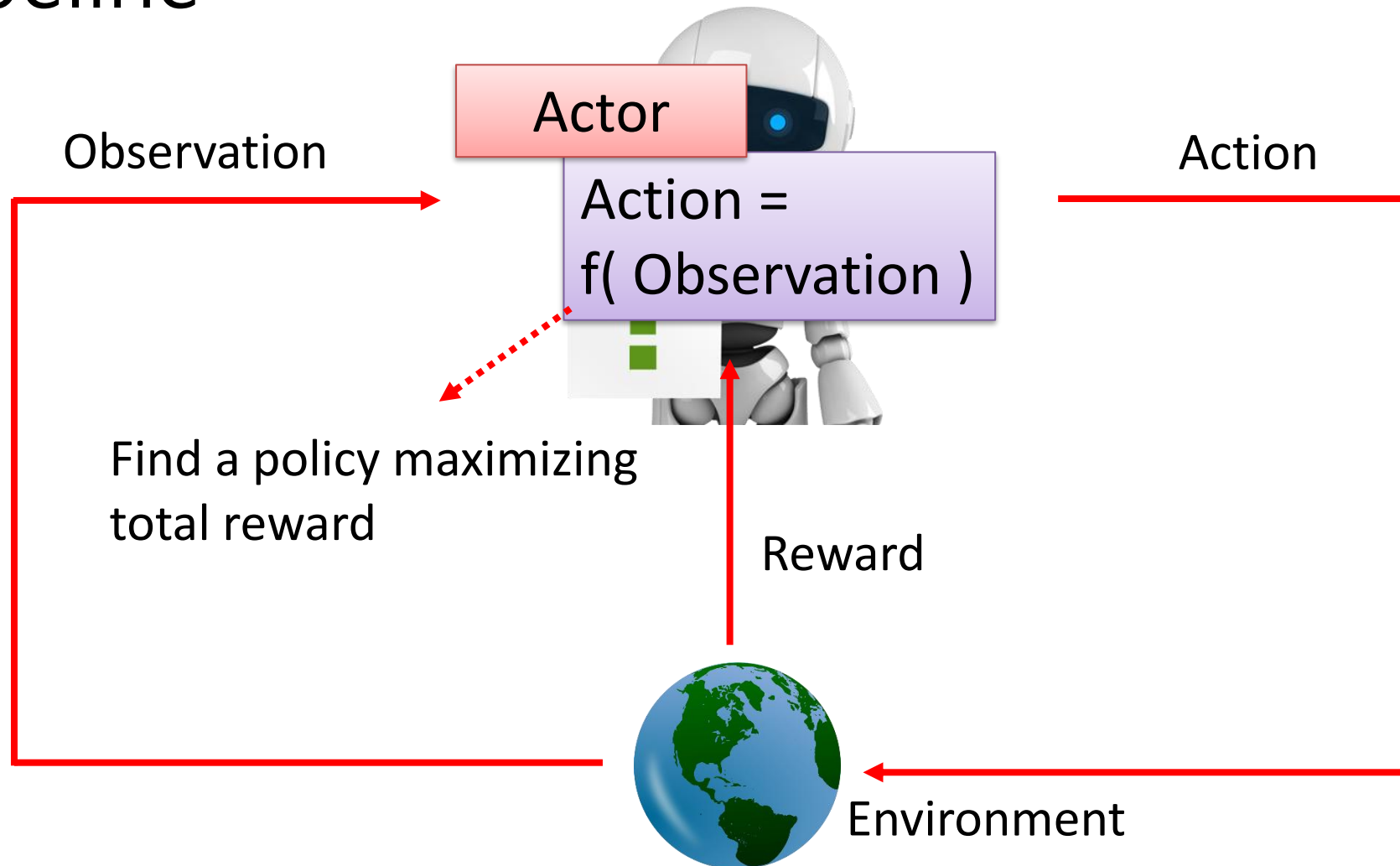
Do not know what is good/bad



Reinforcement Learning



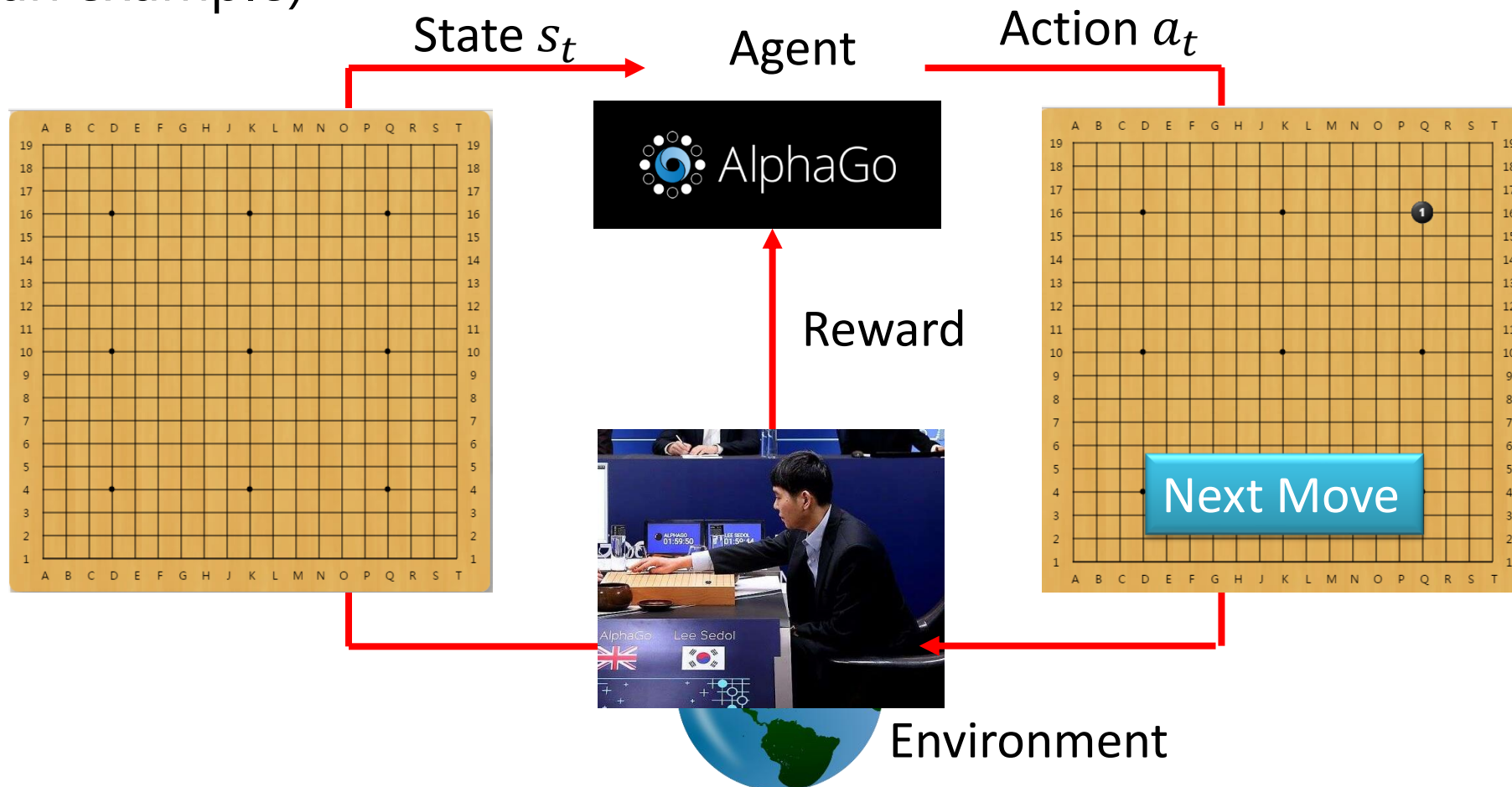
RL pipeline



Reinforcement Learning



RL pipeline (Go as an example)



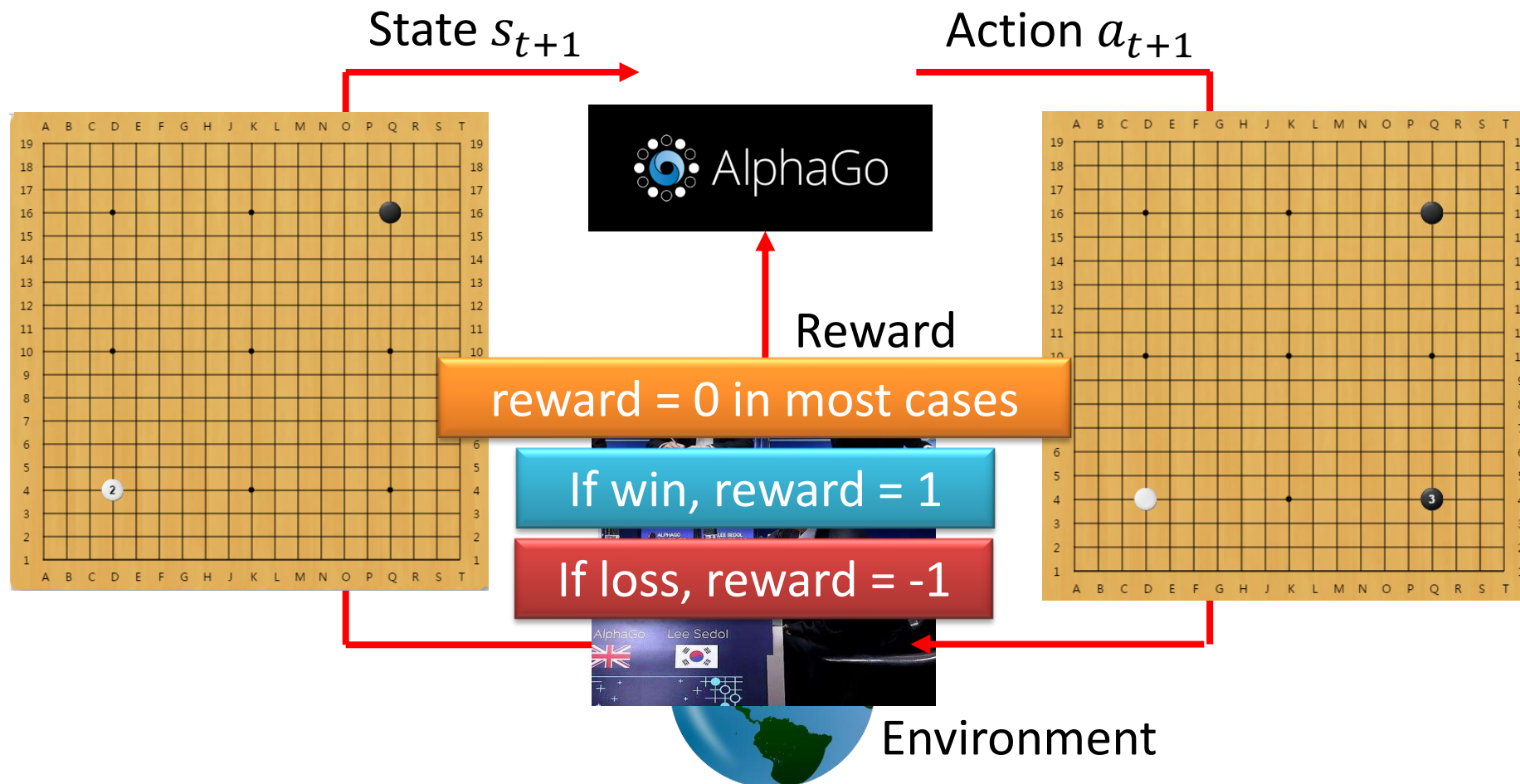
Reinforcement Learning



RL pipeline

(Go as an example)

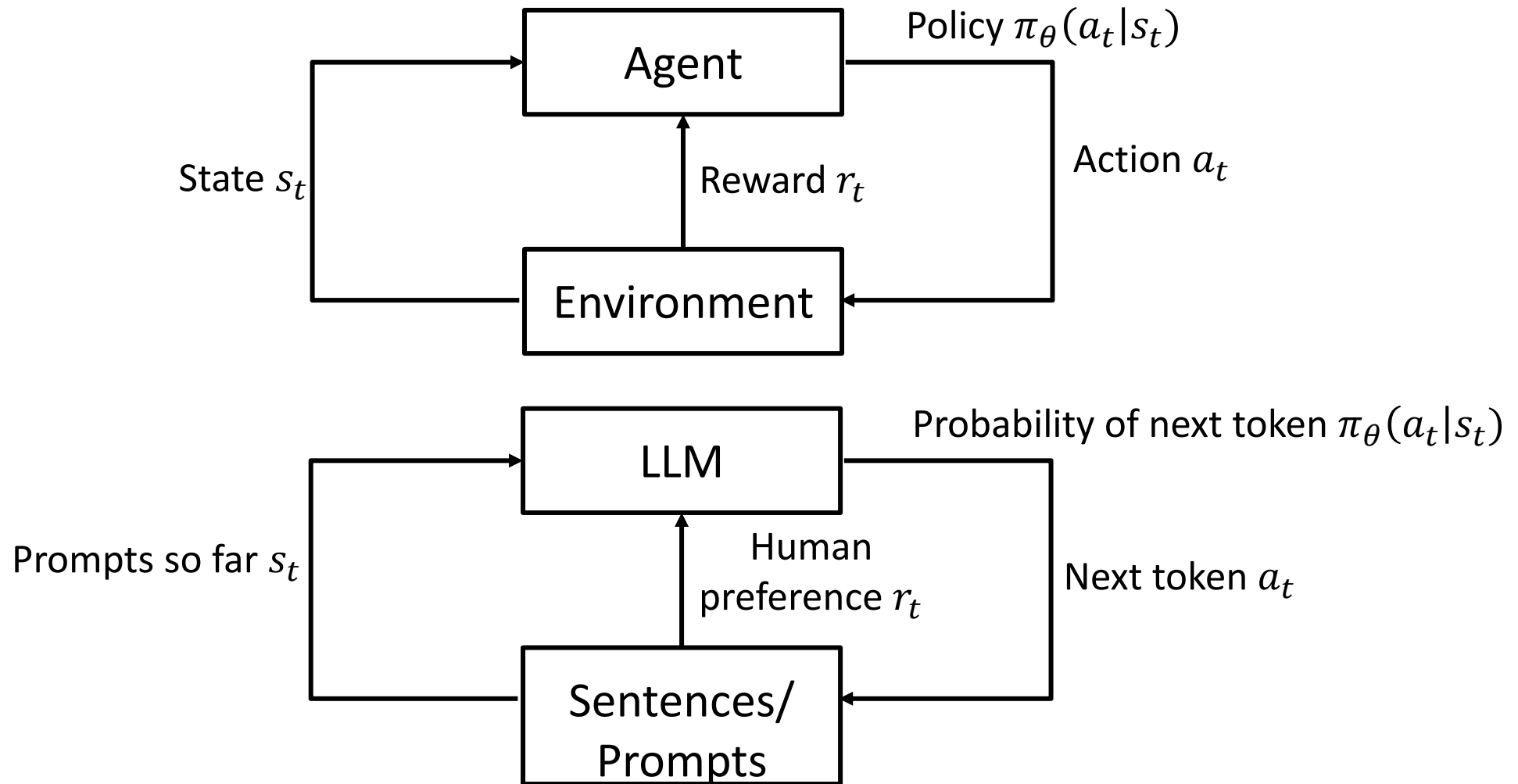
Explore an actor maximizing expected reward.



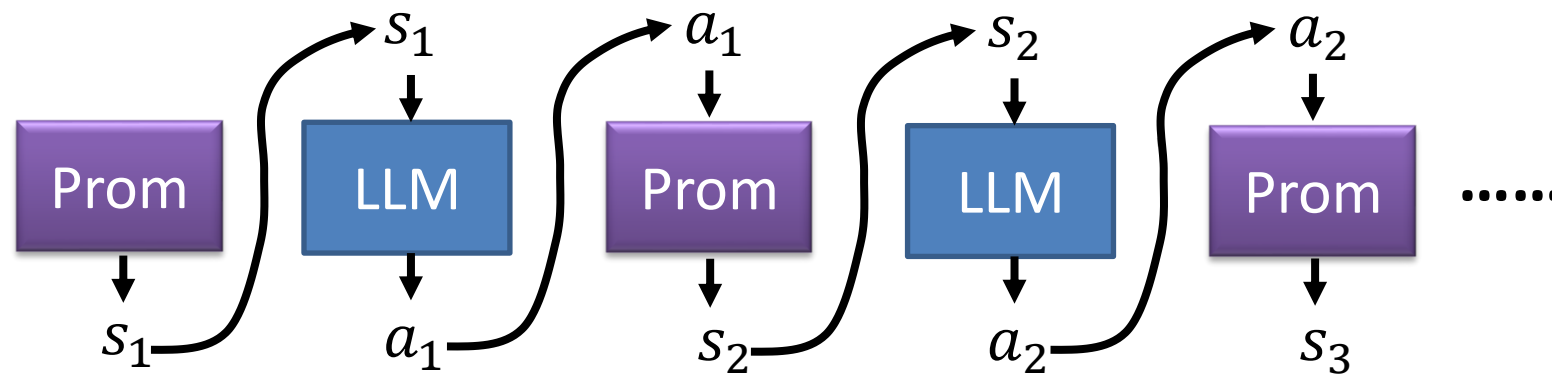
Reinforcement Learning



Agent vs LLM



Reinforcement Learning



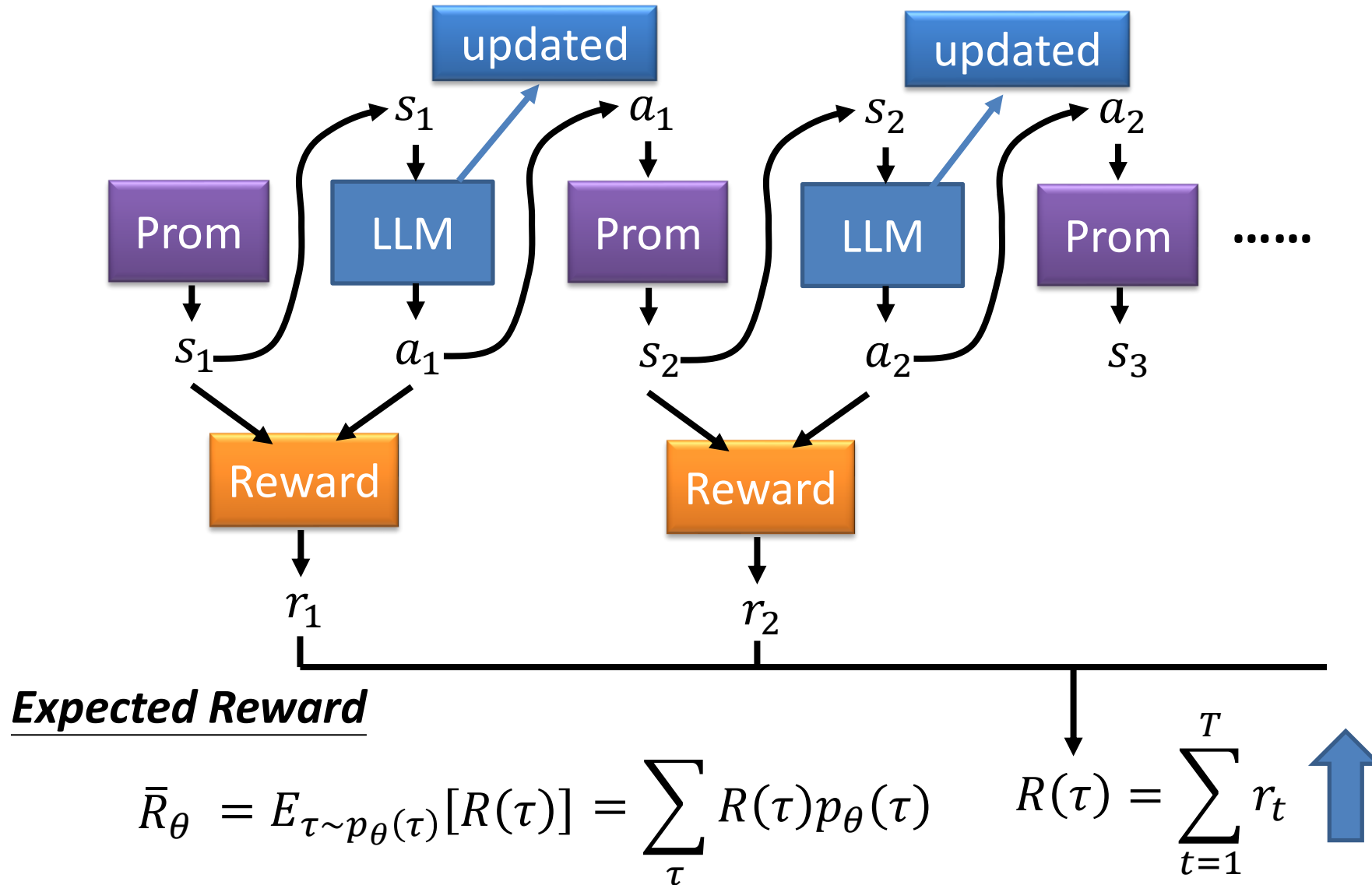
Sentence $\tau = \{s_1, a_1, s_2, a_2, \dots, s_T, a_T\}$

$p_{\theta}(\tau)$

$$= p(s_1)p_{\theta}(a_1|s_1)p(s_2|s_1, a_1)p_{\theta}(a_2|s_2)p(s_3|s_2, a_2) \dots$$

$$= p(s_1) \prod_{t=1}^T p_{\theta}(a_t|s_t)p(s_{t+1}|s_t, a_t)$$

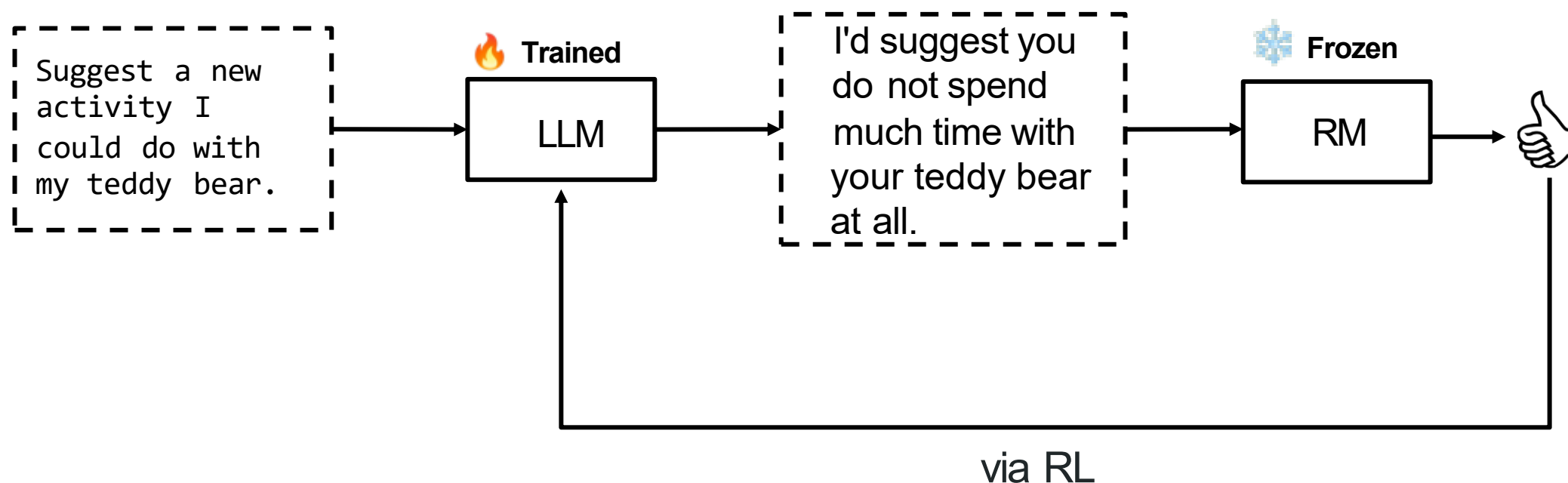
Reinforcement Learning



Reinforcement Learning



Idea. Change weights of LLM to penalize bad answers and promote good answers via **Reinforcement Learning** using the **Reward Model**.



Reinforcement Learning



Objective function optimizes for **higher rewards without going too far** from the **base model**

Base Model. Initialized at SFT model

Training. Change weights of policy (LLM) via objective function:

$$J(\theta) = \boxed{\text{Maximize rewards}} + \boxed{\text{Don't deviate too much from base model}}$$

Avoid **"reward hacking"** + training instability

Reinforcement Learning



Policy Gradient

$$\nabla \bar{R}_\theta = ?$$

$$\nabla \bar{R}_\theta = \sum_{\tau} R(\tau) \nabla p_\theta(\tau) = \sum_{\tau} R(\tau) p_\theta(\tau) \frac{\nabla p_\theta(\tau)}{p_\theta(\tau)}$$

$R(\tau)$ do not have to be differentiable

It can even be a black box.

$$= \sum_{\tau} R(\tau) p_\theta(\tau) \nabla \log p_\theta(\tau)$$

$$\nabla f(x) = f(x) \nabla \log f(x)$$

$$= E_{\tau \sim p_\theta(\tau)} [R(\tau) \nabla \log p_\theta(\tau)] \approx \frac{1}{N} \sum_{n=1}^N R(\tau^n) \nabla \log p_\theta(\tau^n)$$

$$= \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} R(\tau^n) \nabla \log p_\theta(a_t^n | s_t^n)$$

Reinforcement Learning



- **Policy gradient** is to directly optimize the policy of a LLM in RL by estimating the gradient of the expected return with respect to the policy parameters.

Sentence

- Each **trajectory** τ^n consists of a sequence of state-action **Token** pairs $(s_1^n, a_1^n), (s_2^n, a_2^n), \dots$
- Total reward $R(\tau^n)$
- $\nabla \bar{R}_\theta$: the gradient of the reward

Given parameters θ

$$\begin{array}{l} \tau^1: (s_1^1, a_1^1) R(\tau^1) \\ \quad (s_2^1, a_2^1) R(\tau^1) \\ \quad \vdots \quad \quad \quad \vdots \\ \tau^2: (s_1^2, a_1^2) R(\tau^2) \\ \quad (s_2^2, a_2^2) R(\tau^2) \\ \quad \vdots \quad \quad \quad \vdots \end{array}$$

only used once

Update
Model

$$\theta \leftarrow \theta + \eta \nabla \bar{R}_\theta$$

$$\nabla \bar{R}_\theta = \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} R(\tau^n) \nabla \log p_\theta(a_t^n | s_t^n)$$

Data
Collection

Reinforcement Learning



On-policy → Off-policy

$$\nabla \bar{R}_\theta = E_{\tau \sim p_\theta(\tau)} [R(\tau) \nabla \log p_\theta(\tau)]$$

- Use π_θ to collect data. When θ is updated, we have to sample training data again.
- Goal: Using the sample from $\pi_{\theta'}$ to train θ . θ' is fixed, so we can re-use the sample data.

Importance Sampling

$$E_{x \sim p}[f(x)] \approx \frac{1}{N} \sum_{i=1}^N \cancel{f(x^i)}$$

x^i is sampled from $p(x)$

We only have x^i sampled from $q(x)$

$$= \int f(x) p(x) dx = \int f(x) \frac{p(x)}{q(x)} q(x) dx = E_{x \sim q} \left[f(x) \boxed{\frac{p(x)}{q(x)}} \right]$$

Importance weight

Reinforcement Learning



On-policy → Off-policy

$$\nabla \bar{R}_\theta = E_{\tau \sim p_\theta(\tau)} [R(\tau) \nabla \log p_\theta(\tau)]$$

- Use π_θ to collect data. When θ is updated, we have to sample training data again.
- Goal: Using the sample from $\pi_{\theta'}$ to train θ . θ' is fixed, so we can re-use the sample data.

$$\nabla \bar{R}_\theta = E_{\tau \sim p_{\theta'}(\tau)} \left[\frac{p_\theta(\tau)}{p_{\theta'}(\tau)} R(\tau) \nabla \log p_\theta(\tau) \right]$$

- Sample the data from θ' .
- Use the data to train θ many times.

Importance
Sampling

$$E_{x \sim p}[f(x)] = E_{x \sim q}\left[f(x) \frac{p(x)}{q(x)}\right]$$

Reinforcement Learning



On-policy → Off-policy

Gradient for update

$$\nabla f(x) = f(x) \nabla \log f(x)$$

$$= E_{(s_t, a_t) \sim \pi_\theta} [A^\theta(s_t, a_t) \nabla \log p_\theta(a_t^n | s_t^n)]$$

$$A^{\theta'}(s_t, a_t)$$

This term is from
sampled data.

$$= E_{(s_t, a_t) \sim \pi_{\theta'}} \left[\frac{P_\theta(s_t, a_t)}{P_{\theta'}(s_t, a_t)} \cancel{A^\theta(s_t, a_t)} \nabla \log p_\theta(a_t^n | s_t^n) \right]$$

$$= E_{(s_t, a_t) \sim \pi_{\theta'}} \left[\frac{p_\theta(a_t | s_t)}{p_{\theta'}(a_t | s_t)} \cancel{\frac{p_\theta(s_t)}{p_{\theta'}(s_t)}} A^\theta(s_t, a_t) \nabla \log p_\theta(a_t^n | s_t^n) \right]$$

$$J^{\theta'}(\theta) = E_{(s_t, a_t) \sim \pi_{\theta'}} \left[\frac{p_\theta(a_t | s_t)}{p_{\theta'}(a_t | s_t)} A^{\theta'}(s_t, a_t) \right] \quad \text{When to stop?}$$

Reinforcement Learning



Proximal Policy Optimization (PPO)

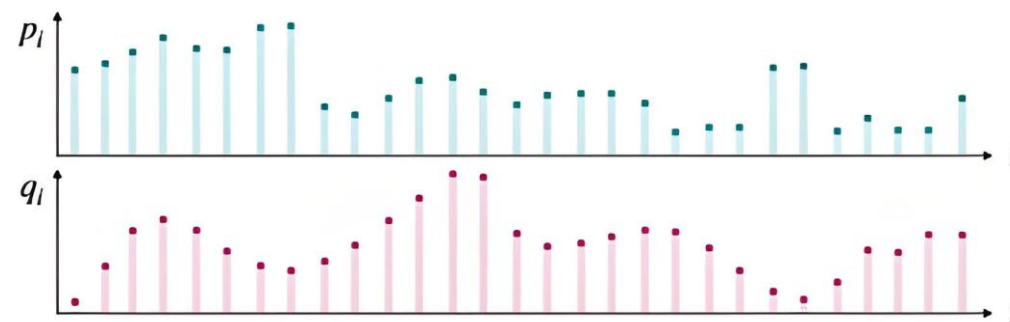
$$J_{PPO}^{\theta'}(\theta) = \underbrace{J^{\theta'}(\theta)}_{\text{Maximize rewards}} - \underbrace{\beta \text{KL}(\theta, \theta')}_{\text{Don't deviate too much from base model}}$$

Maximize rewards

Don't deviate too much
from base model

$$J^{\theta'}(\theta) = E_{(s_t, a_t) \sim \pi_{\theta'}} \left[\frac{p_{\theta}(a_t | s_t)}{p_{\theta'}(a_t | s_t)} A^{\theta'}(s_t, a_t) \right]$$

$$\text{KL}(P||Q) = \sum_{i=1}^n p_i \log \left(\frac{p_i}{q_i} \right)$$

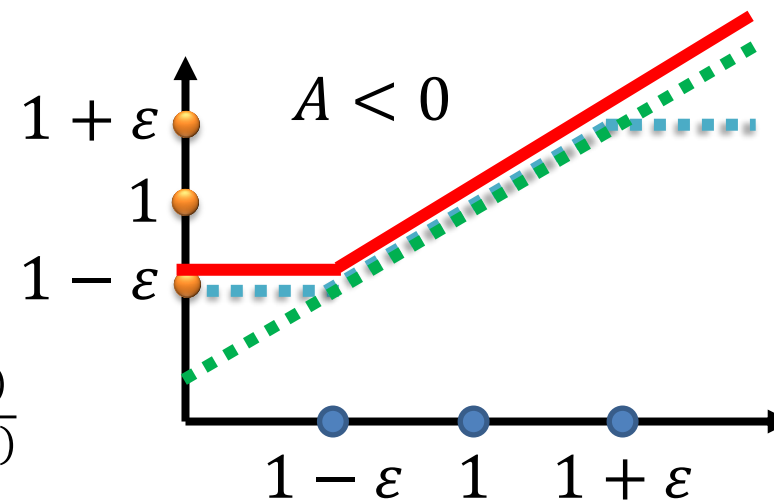
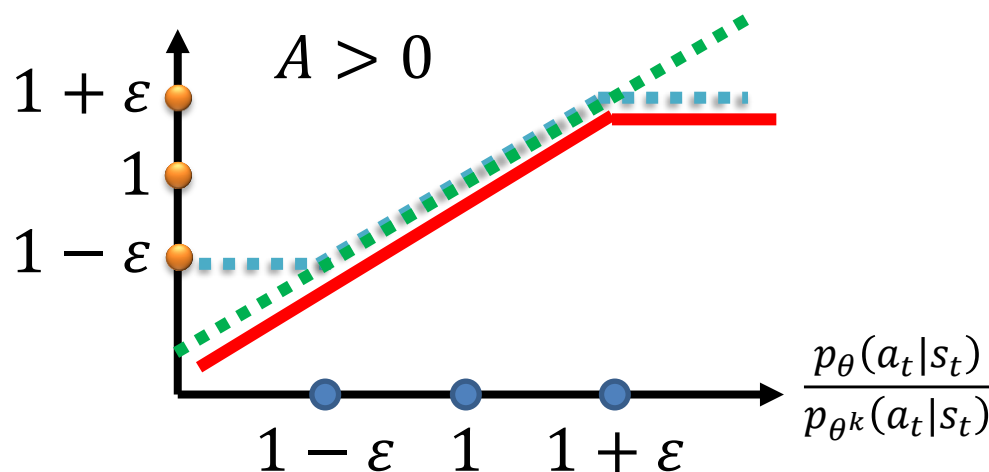


Reinforcement Learning



PPO-Clip

$$J_{PPO2}^{\theta^k}(\theta) \approx \sum_{(s_t, a_t)} \min \left(\frac{p_{\theta}(a_t|s_t)}{p_{\theta^k}(a_t|s_t)} A^{\theta^k}(s_t, a_t), \right. \\ \left. clip \left(\frac{p_{\theta}(a_t|s_t)}{p_{\theta^k}(a_t|s_t)}, 1 - \varepsilon, 1 + \varepsilon \right) A^{\theta^k}(s_t, a_t) \right)$$



Reinforcement Learning



Proximal Policy Optimization (PPO)

$$J_{PPO}^{\theta'}(\theta) = J^{\theta'}(\theta) - \beta KL(\theta, \theta')$$

$$J^{\theta'}(\theta) = E_{(s_t, a_t) \sim \pi_{\theta'}} \left[\frac{p_{\theta}(a_t | s_t)}{p_{\theta'}(a_t | s_t)} A^{\theta'}(s_t, a_t) \right]$$

TRPO (Trust Region Policy Optimization)

$$J_{TRPO}^{\theta'}(\theta) = E_{(s_t, a_t) \sim \pi_{\theta'}} \left[\frac{p_{\theta}(a_t | s_t)}{p_{\theta'}(a_t | s_t)} A^{\theta'}(s_t, a_t) \right]$$

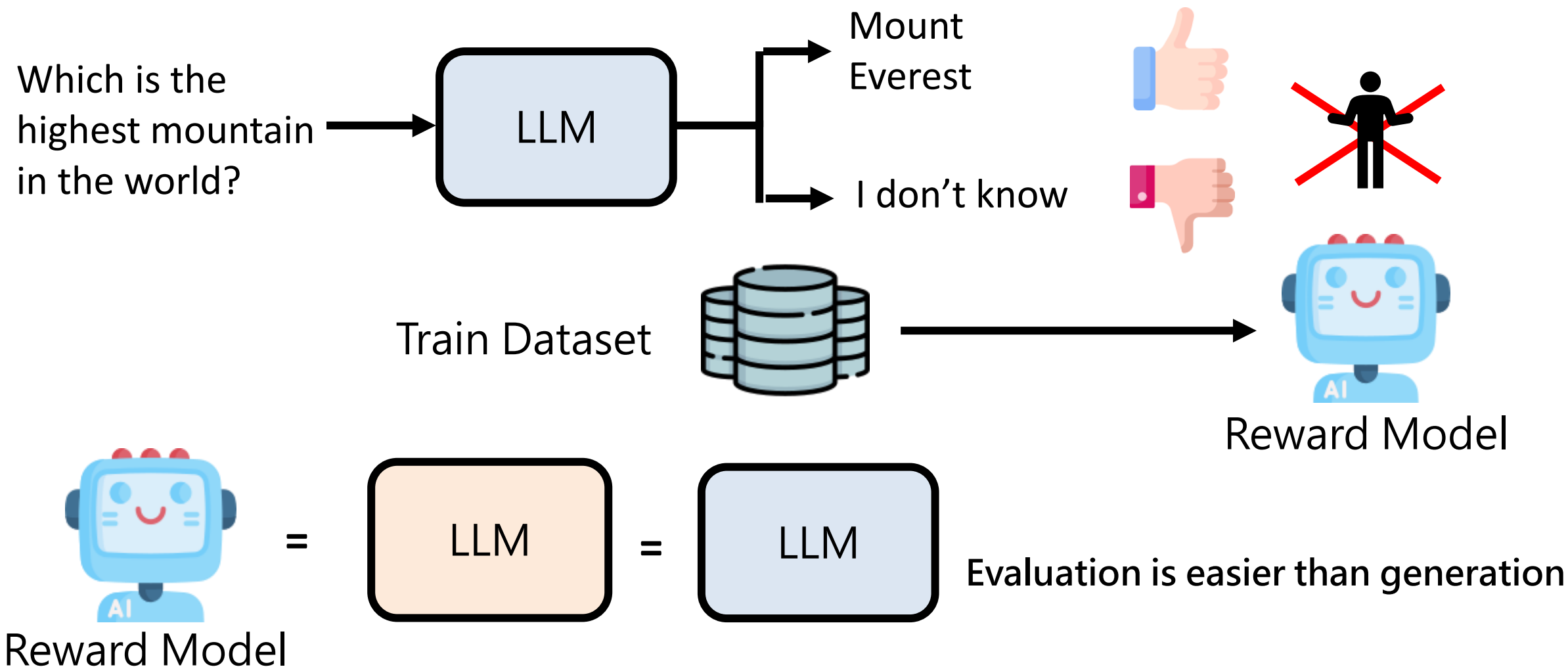
$$KL(\theta, \theta') < \delta$$

Other RL methods: GRPO, DPO, Q-learning, Actor-Critic and many more.....

Reinforcement Learning



RLHF → RLAIF



Summary



Pre-train

Self-supervised
Learning

Datasets

Input: AI is

Output: powerful

Supervised
Fine-tuning

Supervised
Learning

User Input: How are you?

AI output: I'm fine

Full Model-Training

RLHF

Reinforcement
Learning (RL)

User Input: "Which is the highest mountain
in the world? AI:"

Output: "Mount Everest" > "Who can tell me?"

Quiz



1.What is the main difference between SFT and RLHF?

SFT uses _____, while RLHF uses _____.

- A. reward signals / ground-truth labels
- B. ground-truth labels / reward signals
- C. unsupervised learning / semi-supervised learning

Quiz



1.What is the main difference between SFT and RLHF?

SFT uses _____, while RLHF uses _____.

A. reward signals / ground-truth labels

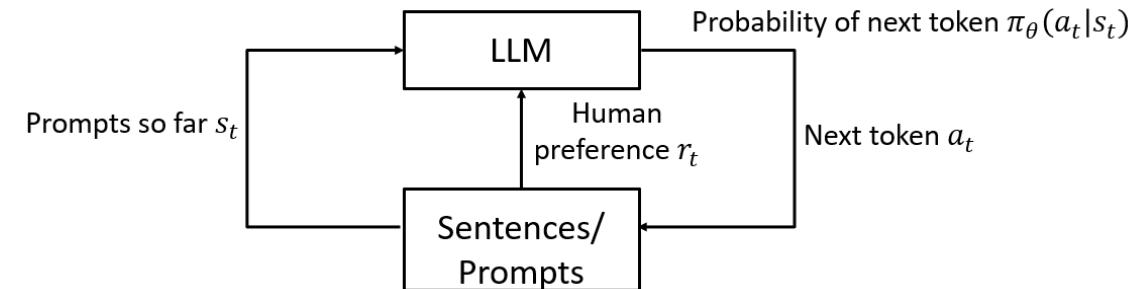
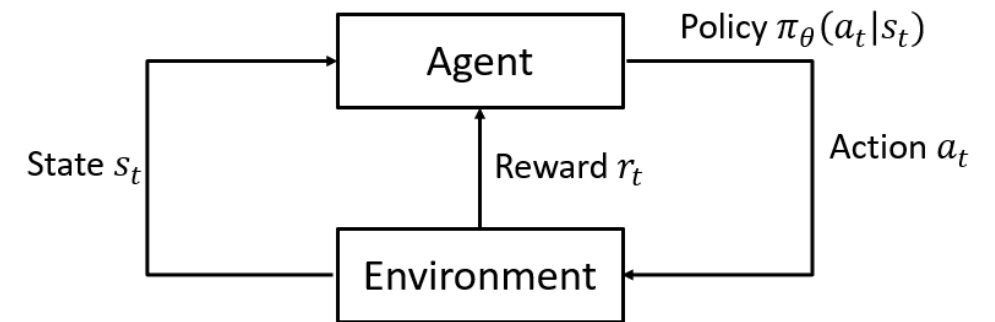
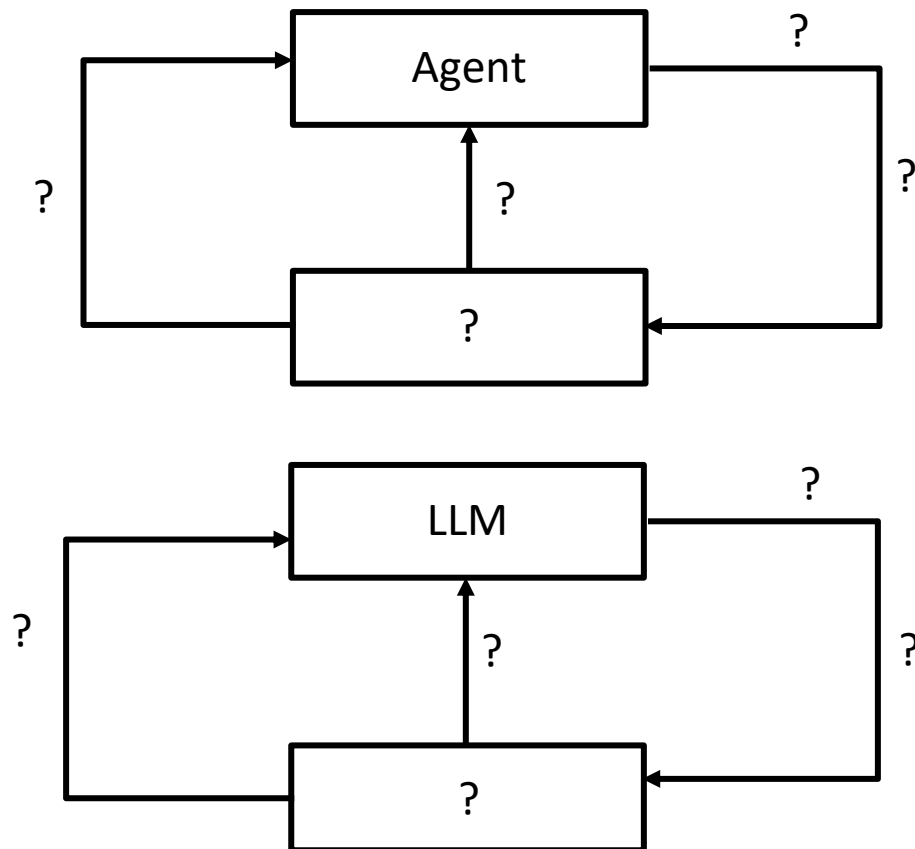
B. ground-truth labels / reward signals

C. unsupervised learning / semi-supervised learning

Quiz



2.Fill in the Reinforcement Learning pipeline.



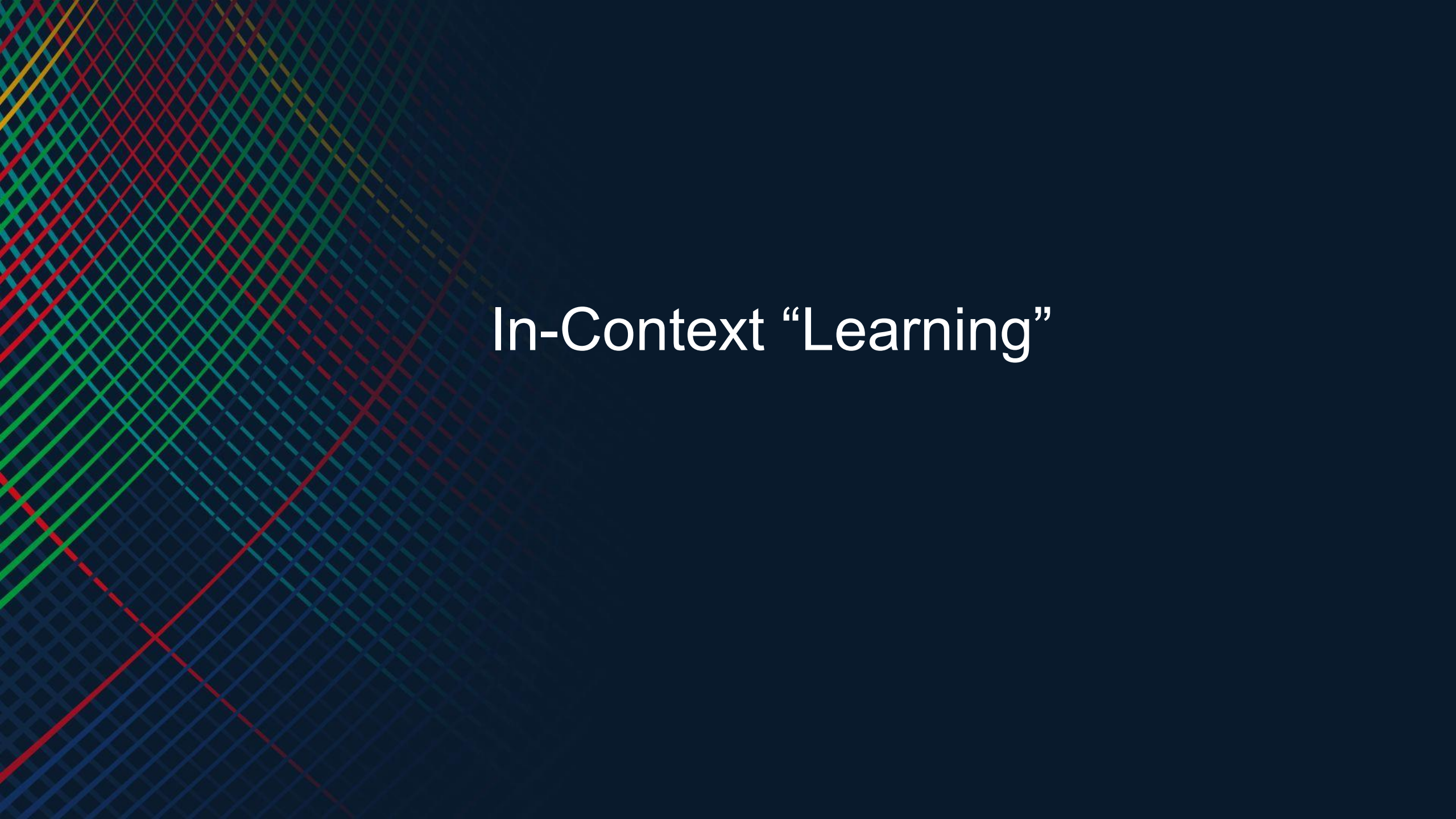


Disadvantages of full model-finetuning for each task

- One model per task is fine for small models, but not for today's big ones.
- **Overfitting / catastrophic forgetting** on small datasets
- Storage of all those model checkpoints (one per task) is expensive.
- During inference time, **it's expensive and inefficient** to have all those models loaded onto GPUs at the same time.

Solutions to Improve Efficiency and Performance

- Avoid finetuning entirely
- In-context learning
- Parameter-efficient finetuning



In-Context “Learning”

In-Context Learning



The three settings we explore for in-context learning:

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1 Translate English to French:  — task description
2 cheese => _____        — prompt
```

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

```
1 Translate English to French:  — task description
2 sea otter => loutre de mer    — example
3 cheese => _____          — prompt
```

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
Translate English to French:  — task description
sea otter => loutre de mer    — examples
peppermint => menthe poivrée — 
plush girafe => girafe peluche — 
cheese => _____          — prompt
```

In -context Learning:

- Input/output pairs from a **training dataset are given in the prompt**
- The model infers the task pattern from the context **without updating its parameters.**
- It performs a form of **implicit reasoning or pattern matching**

In-Context Learning



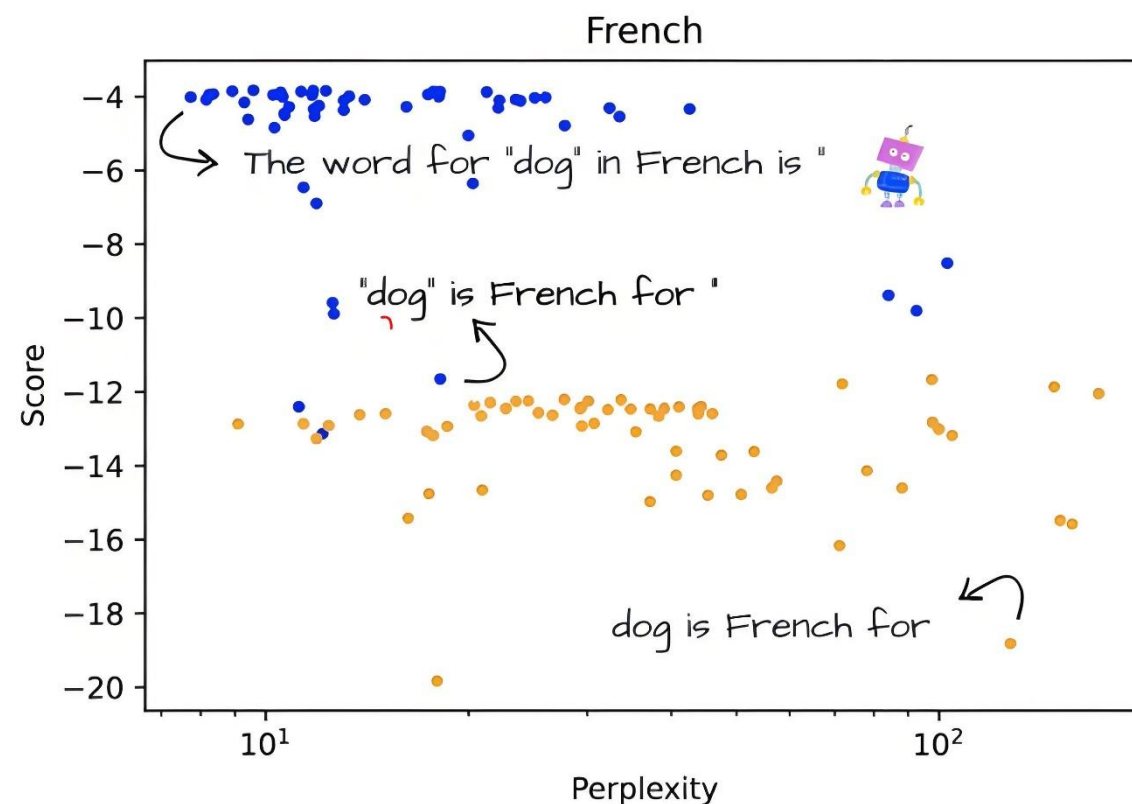
- **Setup:** zero-shot learning

Question: if we evaluate the model multiple times keeping everything fixed except for the prompt, do we always get the same results?

Prompt	Accuracy
What is this piece of news regarding?	40.9
What is this article about?	52.4
What is the best way to describe this article?	68.2
What is the most accurate label for this news article?	71.2

- **Question:** how can we pick a good prompt?

Answer: pick the prompt with the lowest perplexity (highest likelihood) under the model!



In-Context Learning



In-context learning can be sensitive to...

1. the order the training examples are presented

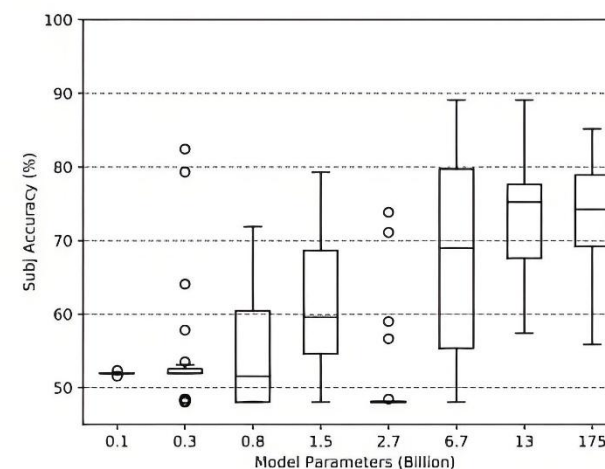
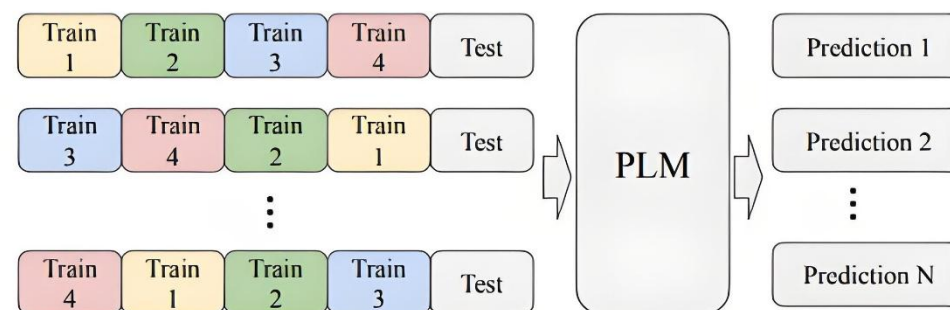


Figure 1: Four-shot performance for 24 different sample orders across different sizes of GPT-family models (GPT-2 and GPT-3) for the SST-2 and Subj datasets.

In-Context Learning



In-context learning can be sensitive to...

2. the balance of labels (e.g. positive vs. negative)

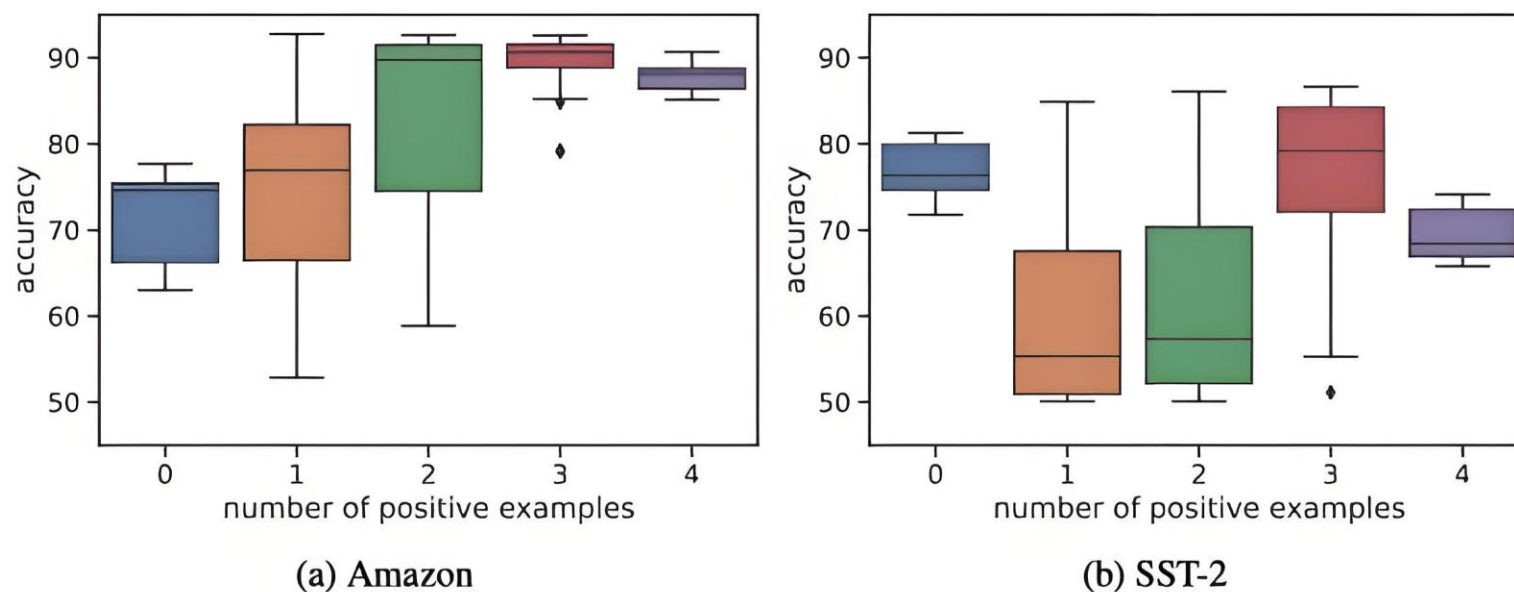


Figure 3: Accuracies of Amazon and SST-2 with varying **label balance** (number of positive examples in demonstration), across 100 total random samples of 4 demonstration examples.

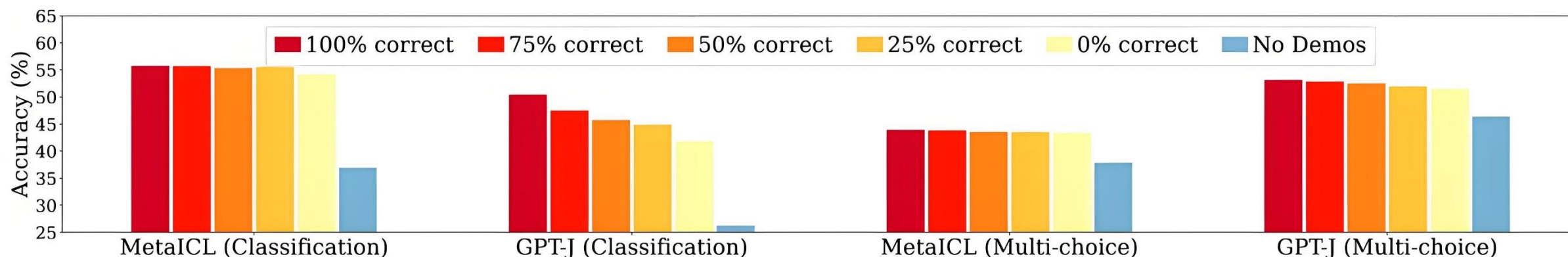
In-Context Learning



Few-shot In-context Learning

You would expect these to be important...

**A. whether or not the training examples have the true label
(as opposed to a random one)**



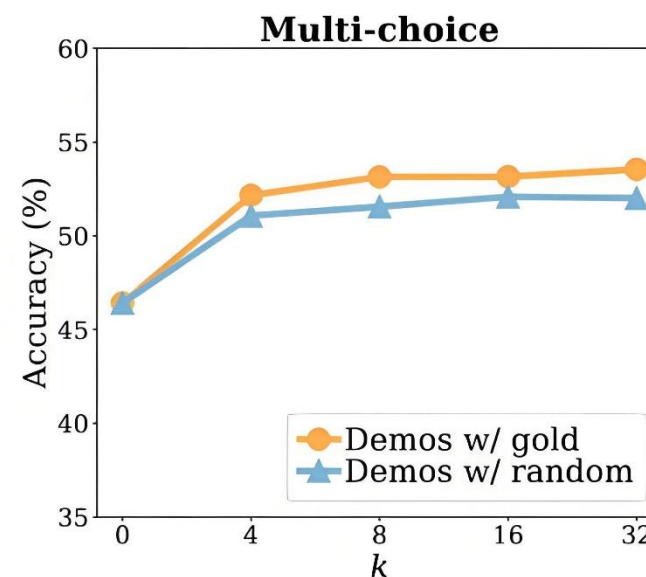
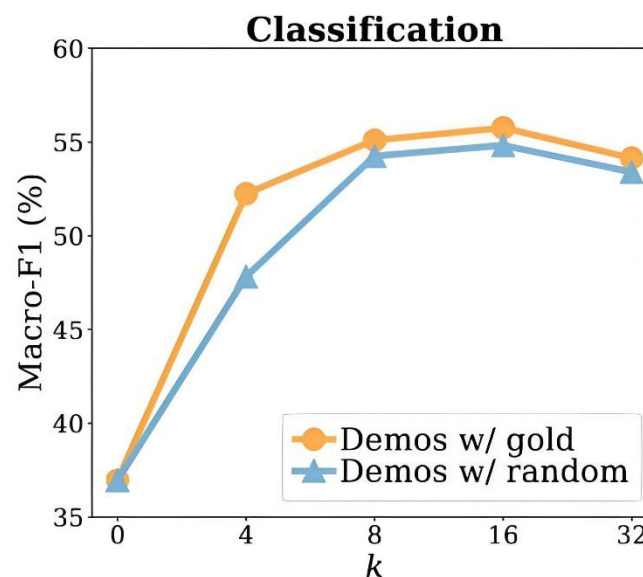
In-Context Learning



Few-shot In-context Learning

You would expect these to be important...

- A. whether or not the training examples have the true label
(as opposed to a random one)
- B. having more in-context training examples ...**
but it's not always the case





Chain-of-Thought Prompting

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✓



Chain-of-Thought Prompting

(c) Zero-shot

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: The answer (arabic numerals) is

(Output) 8 ✗

(a) Few-shot

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A:

(Output) The answer is 8. ✗

(d) Zero-shot-CoT (Ours)

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: **Let's think step by step.**

(Output) *There are 16 balls in total. Half of the balls are golf balls. That means that there are 8 golf balls. Half of the golf balls are blue. That means that there are 4 blue golf balls.* ✓

(b) Few-shot-CoT

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A:

(Output) *The juggler can juggle 16 balls. Half of the balls are golf balls. So there are $16 / 2 = 8$ golf balls. Half of the golf balls are blue. So there are $8 / 2 = 4$ blue golf balls. The answer is 4.* ✓



Chain-of-Thought Prompting

- Asking the model to reason about its answer can improve its performance for few-shot in-context learning
- **Chain-of-thought** prompting provides such reasoning in the in-context examples

	MultiArith	GSM8K
Zero-Shot	17.7	10.4
Few-Shot (2 samples)	33.7	15.6
Few-Shot (8 samples)	33.8	15.6
Zero-Shot-CoT	78.7	40.7
Few-Shot-CoT (2 samples)	84.8	41.3



Thank you!



SUSTech

Southern University
of Science and
Technology